



ANDERSON SERANGOON JUNIOR COLLEGE
JC1 Promotional Examination 2024
Higher 2

COMPUTING

9569/02

Paper 2 (Lab-based)

9 October 2024 (Wednesday), 2 – 4.30 p.m.

2 hour 30 minutes

Additional Materials: Electronic version of `PARTICIPANTS.txt` data file
 Electronic version of `STUDENT.csv` data file
 Electronic version of `LATE.csv` data file
 Insert Quick Reference Guide

READ THESE INSTRUCTIONS FIRST

Answer **all** questions.

You are not allowed to bring in or take out any pieces of work or materials on paper or electronic media or in any other form.

You are NOT allowed to refer to any existing materials in the laptop or online.

Non-exam approved software are disallowed.

Candidates are strictly prohibited from using any AI tools, including but not limited to cs50.ai, Perplexity and ChatGPT, during the examination.

Do NOT write or make any markings on the Insert Quick Reference Guide.

Approved calculators are allowed.

Save each task as it is completed.

The use of built-in functions, where appropriate, is allowed for this paper unless stated otherwise.

Note that up to **6** marks will be awarded for the use of common coding standards for programming style.

The number of marks is given in brackets [] at the end of each question or part question.
The total number of marks for this paper is 85.

This document consists of **9** printed pages and **3** blank pages.

1 Name your Jupyter Notebook as:

TASK1_<your name>.ipynb

The task is to process a square grid of strings and identify patterns of 3 letters that are consecutive alphabetically (e.g. DEF, ONM).

For instance, given the following square grid of 5 strings:

```

D O O R S
C L O T H
B O U N D
A F T E R
F I R S T
  
```

we can identify 4 sets of 3 letters that are consecutive (ascending or descending).

For each of the sub-tasks, add a comment statement at the beginning of the code using the hash symbol '#', to indicate the sub-task the program code belongs to, for example:

```
In [1]: #Task 1.1
        Program code
```

Output:

Task 1.1

Write program code that prompts the user to input a string of capitalised letters. Based on the length of this string, it should continue to prompt for additional string inputs until a square grid can be formed. For example, if the initial string input is 'DOORS', the program would expect four more lines of string input from the user.

The program is to validate the user's input to ensure that

- all characters are capital English letters (A-Z),
- the length of the first string is at least 3 and the length of each subsequent string matches the length of the first string.

If any string does not meet the conditions, the program should re-prompt the user until a valid input is provided.

The program should create a 2-dimensional list `grid` where each row contains the **ASCII values** of the input string's characters. [6]

Test your program using the square grid below, printing each row of ASCII values on a separate line.

```

DAMPEN
BERETS
COFFEE
BEGGAR
DEFEND
WEIGHT
  
```

[1]

Task 1.2

Write program code that counts the total occurrences of three consecutive letters (either in ascending or descending order) that appear horizontally, vertically or diagonally in the given square grid from **Task 1.1**.

Output the result with an appropriate statement.

[7]

Save your Jupyter Notebook for Task 1.

2 Name your Jupyter Notebook as:

TASK2_<your name>.ipynb

A computer game is written using object-oriented programming. It is set on a battlefield, where characters can move around, constrained to the battlefield's boundaries. Two classes are to be implemented: a basic `Soldier` class and a more specialised `Spartan` class.

The class `Soldier` contains three properties:

- `name`: a string that stores the name of the soldier
- `x_pos`: an integer representing the x-coordinate of the soldier
- `y_pos`: an integer representing the y-coordinate of the soldier

It also has the following methods:

- a constructor that initialises `name`, `x_pos` and `y_pos`
- `get_x()` and `get_y()` that return the x- and y-coordinates respectively
- `increment_x()` and `increment_y()`, where each take an integer as a parameter and add this to the current x- or y-coordinate. If the new value exceeds 5000, it is limited to 5000. If the new value is below 0, it is limited to 0.
- a method `move()` that takes a string parameter: "up", "down", "left" or "right".

The table shows the change each direction will make to the coordinates for a soldier:

Direction	Change
up	y-coordinate + 5
down	y-coordinate - 5
left	x-coordinate - 5
right	x-coordinate + 5

The method calls an appropriate method with an integer value. If the direction provided is invalid, it should instead print an output: "Invalid direction".

For each of the sub-tasks, add a comment statement at the beginning of the code using the hash symbol '#', to indicate the sub-task the program code belongs to, for example:

```
In [1]: #Task 2.1
        Program code
```

Output:

Task 2.1

Write program code for the `Soldier` class.

[6]

Task 2.2

The class `Spartan` inherits from the `Soldier` class. The constructor accepts an additional parameter `skills`, which is a list containing unique skills that the Spartan possesses. Consequently a new method needs to be defined to display the skills of the Spartan.

Another difference is that the `move()` method for `Spartan` should trigger a change in x-coordinate by ± 8 for right and left directions and a change in y-coordinate by ± 9 for up and down directions.

Write program code for the `Spartan` class, demonstrating the concepts of inheritance and polymorphism.

[5]

Task 2.3

Write program code to:

- declare an instance of a `Spartan` named Jeremy with skills of swordsmanship, archery and spear throwing, and having a starting position at $(x, y) = (888, 999)$
- display the list of skills of the Spartan
- prompt the user to input the direction of the Spartan's move
- call the appropriate method to move the Spartan and output its position in the format:
`"<name>'s position is (<x_pos>, <y_pos>)."`

[4]

Save your Jupyter Notebook for Task 2.

3 Name your Jupyter Notebook as:

`TASK3_<your name>.ipynb`

There is a current affairs quiz web page that takes in submissions over the months of September and October 2024. Each time a person visits the web page, a set of 10 randomly selected questions will be issued. After the person completes the quiz, a new record is appended to a text file `PARTICIPANTS.txt` with tab-separated values.

Each record consists of:

- timestamp (date and time the quiz was completed)
- MD5 hashed version of the participant's NRIC
- binary string with 10 characters, indicating whether each question was answered correctly (1 for correct, 0 for incorrect).

The first two records are as follows. The quiz scores for these two participants are 9 and 8.

2024-09-01 05:26:45	a540a4015aab6b68b5c5d9ff9ef56d0c	1111101111
2024-09-01 06:24:22	b4fba14520a965f7c1feb09cb182ba42	0111111101

The task is to process these records.

For each of the sub-tasks, add a comment statement at the beginning of the code using the hash symbol '#', to indicate the sub-task the program code belongs to, for example:

```
In [1]: #Task 3.1  
Program code
```

Output:

Task 3.1

Write program code to:

- read the data from `PARTICIPANTS.txt` into a 2-dimensional list `participants`
- convert the binary string in the last column of each record into an integer score (total number of correct answers)
- output the last 5 rows of `participants` on separate lines.

Each row in `participants` contains only 3 entries: the timestamp, the hashed NRIC and the participant's score for the quiz. [4]

Task 3.2

Write a function `merge_sort(arr)` that implements the merge sort algorithm to sort a list in ascending order.

Apply this function to the 1-dimensional list of MD5 hashed NRICs from `participants` and store the result in a list named `hash_sorted`.

Display the first 6 hashed NRICs in the `hash_sorted` list. [7]

Task 3.3

Write a **recursive** function `bin_search(arr, target, low, high)` that performs a binary search on a sorted list to find a **given** `target`.

The function should return 1 if `target` is found, and -1 otherwise.

Test the function with the `hash_sorted` list using two suitable test cases and print the number of function calls (including the initial call) for each test case. [7]

Task 3.4

Write program code to create another 2-dimensional list `processed` from `participants` where:

- all records with scores less than 8 are removed
- for each participant, if he or she completed multiple quizzes on the same day, only one record with the highest score should be retained for that participant that day.

Output the processed data to a file `PROCESSED.txt` with tab-separated values.

Submit this text file as evidence. [6]

Save your Jupyter Notebook for Task 3.

4 Name your Jupyter Notebook as:

TASK4_<your name>.ipynb

A teacher wants to keep track of instances where his students arrive late to college.

To efficiently manage this data, he wants to create a database that records both the students' information and latecoming incidents. Two tables with the specified fields are to be created:

Student:

- `stu_id` – unique student ID, for example, 24076
- `name` – the student's full name
- `cg` – the class group that the student is in, for example, 25/13
- `handphone` – the handphone number of the student, stored as a string

Latecoming:

- `date` – date of student's latecoming incident, for example, 2024-09-13
- `stu_id` – student ID of the latecoming student
- `reason` – reason for latecoming, as cited by the student

For each of the sub-tasks 4.1 to 4.3, add a comment statement at the beginning of the code using the hash symbol '#', to indicate the sub-task the program code belongs to, for example:

```
In [1]: #Task 4.1
        Program code
```

Output:

Task 4.1

Write a Python program that uses SQL code to create the database `LIBRARY.db` with the two tables given, defining the appropriate primary and foreign keys for each table. [5]

Task 4.2

The teacher has downloaded data of students in his classes from the ASR Portal as `STUDENT.csv`. Additionally, he also received recent student latecoming records from the Dean, stored in another file `LATE.csv`.

Write a Python program to read in the data from each file and store them in the database. [4]

Task 4.3

Write a function `filter_latecomers` that takes the parameters `cg`, `min_latecomings`, `start_date`, `end_date` and returns a **list of strings**, which are the names (in alphabetical order) of students who belong to the specified class group and have been late at least the specified number of times within the date range.

The function should use SQL code to extract and filter the relevant data.

Test the function by printing the list of students from class 25/13 who have at least 2 latecoming incidents in August 2024. [6]

Save your Jupyter Notebook.

Task 4.4

Create a **single-page** web application that allows the teacher to search for a student's latecoming records.

- A user that visits the page will be greeted with a form where he can input the name of a student to search for.
- If the student is found within the latecoming records, a table with orange cell borders should be displayed, showing the date and the latecoming reason for each latecoming record of the student.
- The records are to be sorted by date, with the latest latecoming record as the first row of data. An appropriate header row should be included in the table.
- Above the table, there should also be two lines of text, one line with the student's name and another showing the total number of latecoming instances.
- If no records are found for the student, the text "There are no latecoming records found for <student name>." should be shown on the page instead. No tables should be displayed.

The background colour of the page should be set as #2F8A7B. All texts on the page should be white and in Arial font.

Save the relevant files and subfolders in a folder named:

TASK4_4_<your name>.

[11]

